Connect Four Game Move Prediction Using Machine Learning

Sarah Aljuhani , Dimah Alotaibi , Yara Alanazi

***Abstract*—In this project, multiple supervised machine learning models were implemented to predict the next optimal move in the popular Connect Four game. The goal was to compare the performance of several algorithms and identify the model that achieved the highest accuracy. The models developed include Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, Neural Network, LightGBM, and Convolutional Neural Network. The dataset was analyzed and preprocessed, with additional feature engineering applied to improve performance. Each model was evaluated on the validation dataset using accuracy and confusion matrix analysis. Among the tested models, the Convolutional Neural Network achieved the best results, showing the highest accuracy of 68.88% and somewhat strong generalization performance.**

***Index Terms*—Machine Learning, Decision Tree, Random Forest, LightGBM, XGBoost, Logistic Regression, Neural Network, Gradient Boosting, Convolutional Neural Network**

I. Introduction

THe Connect Four game is a well-known game commonly played by multiple individuals worldwide. In this game, two players take turns to drop their different colored coins in a two-dimensional grid. The first player who forms a vertical, horizontal, or diagonal line using four coins wins. However, if the board is completely filled and no player has created the winning line, then the game is considered to be a draw [1].

In this project, multiple different supervised learning machine learning models, such as Decision Tree, Random Forest, and XGBoost, were built to predict the optimal next move for the players depending on their turns. The aim is to explore several machine learning models and feature engineering techniques to determine which will achieve the highest accuracy in predicting the next move. We assumed that the game is played on a 7-column and 6-row grid, in which players take turns to drop their tokens in their desired locations. A dataset was given, containing the flattened version of the board state (p1 to p42), the turn, which indicates whose turn it is to play the game, (1 or -1), and lastly, the target label_move_column, which indicates the optimal move at the given state, and is the column that the model is trying to correctly predict. The dataset was then split into a training set, a validation set, a and test set.

The report is divided into several sections. First, the methodology used to predict the next player move is presented, along with the feature engineering techniques and a description of the selected models. Next, the experimental results section presents the results of the selected models. Then, the hyperparameter tuning section describes how the models were tuned to improve their performance. The discussion section highlights the main findings of the experiments. Lastly, the conclusion section summarizes the report and the work done.

II. Methodology

In this section, the methodology used to predict the next player move is described. The methodology includes feature engineering techniques we have tried and the different machine learning models implemented, including baseline models, such as Decision Tree, and more advanced algorithms, such as XGBoost.

A. Data preprocessing and Feature Engineering

In this project, three datasets were given. The first dataset is the ‘train.csv’, which, as the name suggests, was used to train the various machine learning models. The second dataset is the ‘val.csv’ dataset, which was used to validate our models and evaluate their performance. The last dataset given is the ‘test.csv’, however, it is only used to make predictions and submit to the competition, as no labels are given.

The training dataset was analyzed to understand the relationships between the data values. It was observed that the dataset is highly unbalanced, strongly favoring the optimal next column move of 3. This indicates the need for exploring machine learning models and feature engineering techniques that can help capture such complex patterns.

The given datasets had values in the range of [-1,1] and [0,1]. Therefore, since the data values were already scaled, when we experimented with standardization, it had little to no effect on our models results. Thus, we decided not to apply it to prevent it from affecting our feature engineering techniques.

Additionally, feature engineering was performed by converting the ‘turn’ numerical feature, where -1 represents the opponent player and 1 represents the current player, into binary features. However, after experimentation, this had no effect on our machine learning models, including XGBoost, Random Forest, and Logistic Regression.

An additional feature engineering step was applied only on the advanced models, such as XGBoost, Neural Networks, LightGBM, Gradient Boosting, and Convolutioanl Neural Network to enhance model learning. The 42 board position features (p1–p42) were multiplied by the ‘turn’ value to reflect the current player, after which the ‘turn’ column was removed. This transformation helped the models better capture the board–player relationship and slightly improved the accuracy of the Neural Network and LightGBM models.

B. Decision Tree Model

The first baseline model we have implemented is the Decision Tree model. In order to create a decision tree model that has high accuracy, simple systematic steps were followed. First, the model was created using the Decision Tree Classifier function, setting the random state to 42 and the class weight to 'balanced' because the dataset is highly unbalanced. Finally, the model was trained using the given training dataset, and predictions were made using the validation dataset.

C. Logistic Regression Model

The second model implemented was a Logistic Regression model enhanced with polynomial features of degree 2 to allow the model to capture nonlinear patterns in the data. To ensure reproducibility, we set a fixed `random_state`, and we increased `max_iter` to guarantee model convergence during training. Because the dataset showed high class imbalance, we used `class_weight='balanced'` so the model assigns appropriate weights to minority classes. After that, we trained the model on the training data, and we evaluated its performance on the validation set by calculating the accuracy and plotting the confusion matrix.

D. Random Forest Model

The third model we implemented was the Random Forest Classifier, a collective method that combines multiple decision trees which predicts using majority voting [2]. Each tree is trained on a random sample of the training data, which helps reduce variance and improves generalization compared to a single decision tree. First, we trained the Random Forest model on the training dataset and evaluated it on the validation dataset while testing a small set of configurations for `n_estimators` and `max_depth`. Then, the best setting was selected based on the validation accuracy. Finally, the model was retrained on the combined training and validation data. This model was chosen for its ability to reduce overfitting and improve overall prediction stability compared to single-tree models [2].

E. Gradient Boosting Model

Moving on to advanced models, we implemented Gradient Boost to try to improve the prediction results. This model improves performance by building a sequence of small decision trees, where each tree focuses on correcting the errors made by the previous ones [3]. We configured the model with a set of hyperparameters that help balance complexity and generalization, such as controlling the number of trees, the learning rate, and the depth of each tree. After training the model on the training data, we evaluated its performance using accuracy and the confusion matrix on the validation set.

F. Extreme Gradient Boosting (XGBoost) Model

Furthermore, we implemented Extreme Gradient Boosting (XGBoost) [4] to predict the next player move. XGBoost is an efficient and advanced algorithm for tree boosting. It is a popular machine learning algorithm designed to handle sparse data [4]. First, we started with a baseline XGBoost without tuning its hyperparameters to analyze how well it performs. After evaluating its performance, we tuned its hyperparameters to improve its results, as described in detail in the hyperparameter tuning section. Lastly, after tuning, the model was trained on the training set and evaluated on the validation set.

G. Neural Network Model

Adding to advanced models, we implemented a Neural Network model due to its ability to efficiently capture complex relationships in data [5]. The architecture consists of one input layer that takes the features from the dataset as input. In addition, the architecture consists of three hidden layers, the first containing 128 neurons, the second containing 64 neurons, and the last containing 32 neurons. All of the hidden layers used the rectified linear unit (ReLU) activation function for simplicity and a dropout layer that randomly "drops" neurons to prevent overfitting was integrated after each hidden layer. Lastly, the architecture included an output layer with 7 neurons and a SoftMax function was used for multiclass classification. The model was optimized using the Adam optimizer [6] and was trained on the training dataset with epoch of 120 and batch size of 32.

H. Light Gradient Boosting Machine (LightGBM) Model

Furthermore, we experimented with the dataset on the Light Gradient Boosting Machine (LightGBM) classifier. This model was selected for its efficiency and strong capability in handling multiclass classification tasks [7]. First, we prepared the LightGBM training and validation datasets. Then, we configured the model's hyperparameters for multiclass classification and trained it on the training set while monitoring performance on the validation set. Early stopping was used to reduce overfitting and improve generalization. Finally, we used the best settings to retrain the model on the combined training and validation dataset.

I. Convolutional Neural Network (CNN)

Moreover, to continue with advancements, we implemented a convolutional neural network (CNN) as our final model to further enhance the next move prediction accuracy. CNNs are commonly used to capture important features, especially in images and grid-like topologies [8]. Therefore, in addition to the feature engineering step where the board positions are multiplied by the 'turn' value to reflect the current player, the Connect-Four game board was reshaped into a 6×7 grid representation. Additionally, this grid is converted into three binary channels to resemble an image, where the first channel represents the current player tokens, the second channel represents the opponent player tokens, and the third and last channel represents the empty cell. This is done so the CNN can efficiently learn and understand the grid similar to the way it learns patterns in images. Next, these channels are stacked into a $6 \times 7 \times 3$ tensor, which is then used an input into the CNN.

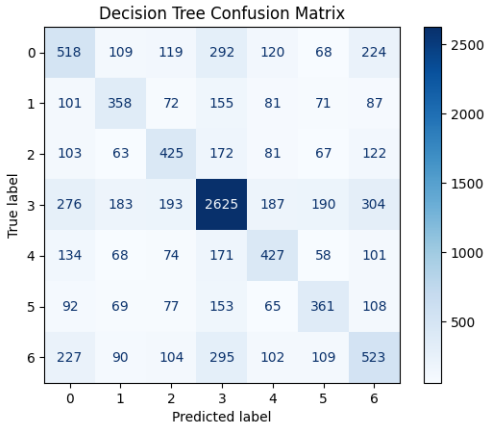

Fig. 1. Confusion matrix of the Decision Tree model.

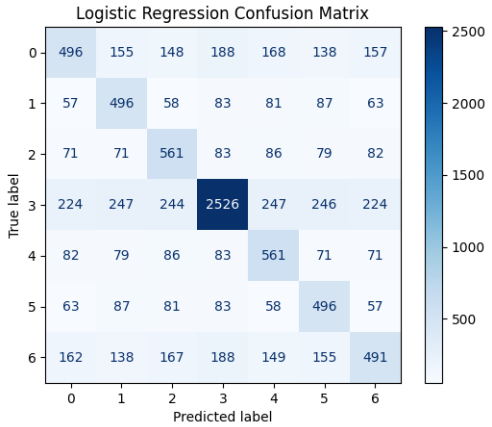

Fig. 2. Confusion matrix of the Logistic Regression model.

The architecture of the CNN consists of four convolutional layers with batch normalization between these layers for a more consistent and faster training [9]. The first two convolutional layers consists of 32 filters of sizes 3×3, with ReLU as the activation function for nonlinearity, and padding that preserves the size of the input. Additionally, L2 regularization is added to reduce overfitting. Next, a max pooling layer is applied to reduce the size of the feature maps while preserving the important information. The next two convolutional layers consists of 64 filters of sizes 3×3 to learn more complex features. Similar to the earlier convolutional layers, it applies ReLU as the activation function, uses padding to preserve the size of the input, and includes L2 regularization to prevent overfitting. After these convolution layers, the feature maps extracted are flattened into a 1-dimensional vector and goes through three dense layers, with dropout, to produce the output probability. The CNN model is then trained using the Adam optimizer [6] and categorical cross entropy as the loss function, as well as a learning rate scheduler that automatically reduces the learning rate by half if the model does not improve. Early stopping is also applied where the model stops learning when the validation loss does not improve.

III. Experimental Results

In this section, a description is given on the results of each of the models implemented, along with their confusion matrix.

The Decision Tree model showed an accuracy of 48.5% on the validation set. Fig. 1 shows the confusion matrix of the Decision Tree model.

The Logistic Regression model with polynomial features of degree 2 showed an accuracy of 52.2% with regularization parameter of 100 when evaluated on the validation set. Fig. 2 illustrates the confusion matrix of the Logistic Regression model.

The Random Forest model achieved an accuracy of 58.9% on the validation set. The confusion matrix, shown in Fig. 3, illustrate that the model performed consistently across most classes, showing better results compared to the Decision Tree and Logistic Regression models.

Moving on to more advanced models, the baseline XGBoost model without any hyperparameter tuning achieved an accuracy of 61%, whereas the XGBoost model modified by tuning its hyperparameters achieved an accuracy of 64.45%. Fig. 4 shows the confusion matrix of the XGBoost model after tuning its hyperparameters.

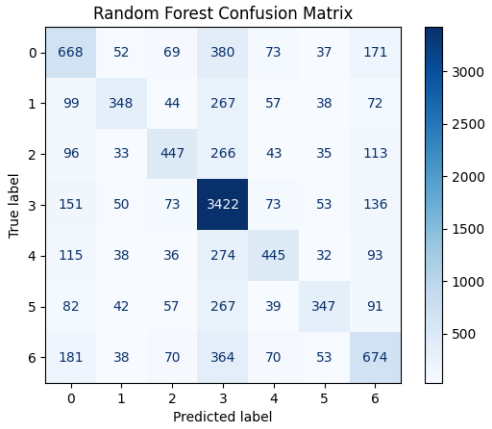

Fig. 3. Confusion matrix of the Random Forest model.

The Gradient Boosting model achieved an accuracy of 62.8% on the validation set. Fig. 5 shows the confusion matrix of the Gradient Boosting model.

The Neural Network model created achieved an accuracy of 59.63%. This was possible after experimenting with different epochs and batch sizes. Fig. 6 shows the confusion matrix of the Neural Network model after training it for 120 epochs with a batch size of 32.

The LightGBM model achieved an accuracy of 64.09% on the validation set. The confusion matrix, shown in Fig. 7, demonstrates that the model performed consistently across most classes, with improved accuracy compared to the Random Forest model. The LightGBM model benefited from the earlier feature engineering step.

Lastly, after experimenting with different L2 regularization values, the final Convolutional Neural Network (CNN) implemented received an accuracy of 68.88% on the validation set. Fig. 8 illustrates the confusion matrix for the CNN.

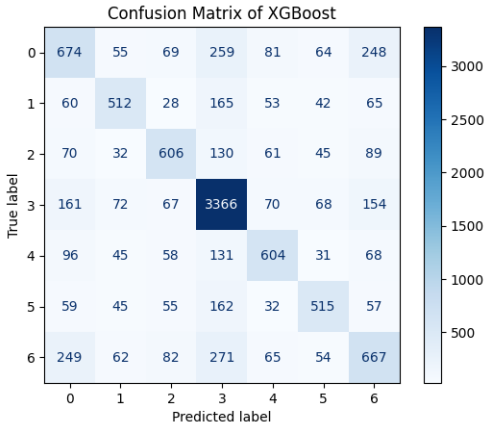

Fig. 4. Confusion matrix of the XGBoost model.

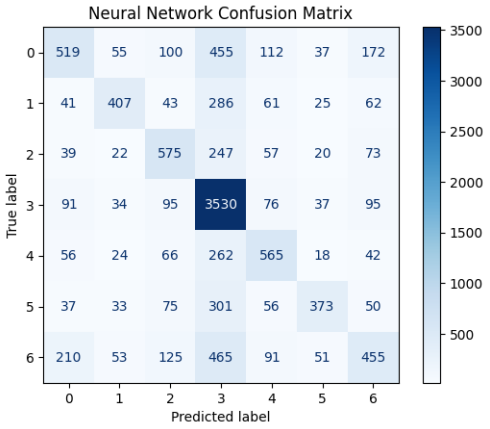

Fig. 6. Confusion matrix of the Neural Network model.

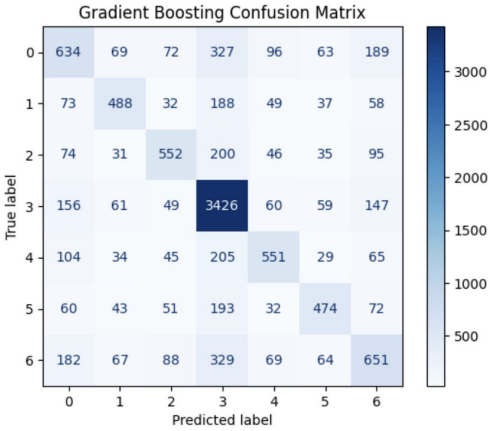

Fig. 5. Confusion matrix of the Gradient Boosting model.

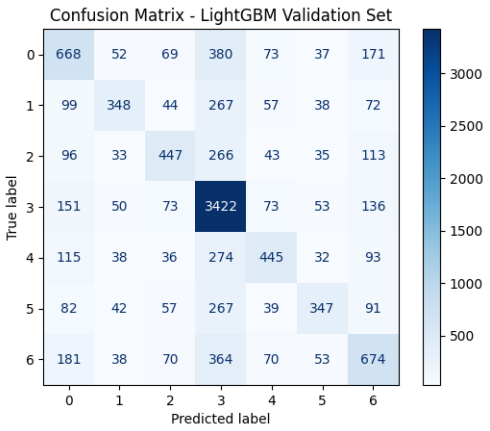

Fig. 7. Confusion matrix of the LightGBM model.

IV. Hyperparameter Tuning

This section provides a detailed description of how the hyperparameters of the models employed were tuned to improve their performance.

A. Decision Tree Hyperparameter Tuning

Starting with the Decision Tree model, we attempted to improve its accuracy by tunning its key hyperparameters, such as `max_depth` that specifies the depth of the tree, and `min_samples_split` that specifies the minimum number of samples to split a node [10]. However, when experimenting with their values, it led to a decrease in the model accuracy. Therefore, we did not apply such hyperparameter tuning in our final decision tree model.

B. Logistic Regression Hyperparameter Tuning

Secondly, we tuned the hyperparameters of the Logistic Regression model. During hyperparameter tuning, we tested several values of the regularization parameter C, including 0.1, 1, 10, and 100. The results show that C = 100 achieved the highest accuracy. We also experimented with different solvers, such as 'sag', but 'lbfgs' provided the most stable and reliable performance. In addition, we increased `max_iter` to 1000 to ensure proper convergence of the model, and we set `class_weight` = 'balanced' to address the class imbalance in the dataset.

C. Random Forest Hyperparameter Tuning

To improve the Random Forest model, we manually tested a small set of hyperparameter settings and selected the best configuration based on validation accuracy. The main tuned parameters were `n_estimators` (number of trees) and `max_depth` (maximum tree depth). The best-performing setting used `n_estimators = 300` with `max_depth = None`, achieving a validation accuracy of 58.9%. The final model was then trained using this configuration with `max_features='sqrt'`, `bootstrap=True`, and `random_state=42`, and it was retrained on the combined training and validation data to generate the final test predictions.

D. Gradient Boosting Hyperparameter Tuning

Our first advanced model is the gradient boosting model. After manually testing with several hyperparameter values, the final model was configured with `n_estimators = 400`, `learning_rate = 0.05`, `max_depth = 6`,

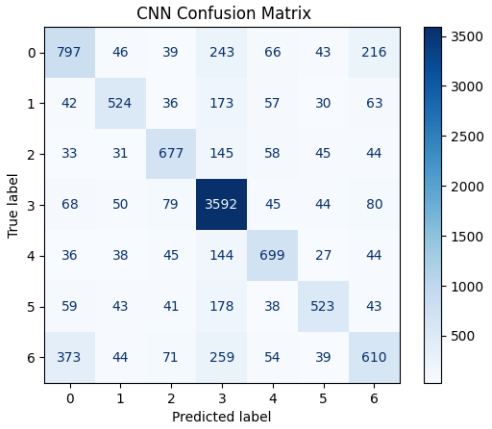

Fig. 8. Confusion matrix of the CNN model.

`subsample = 0.8`, `min_samples_split = 5`, and `min_samples_leaf = 2`. These hyperparameters were selected to maintain an appropriate balance between bias and variance while reducing the risk of overfitting.

E. Extreme Gradient Boosting (XGBoost) Hyperparameter Tuning

To improve the results of the baseline XGBoost, its hyperparameters were tuned utilizing the randomized search algorithm [11]. The set of hyperparameters tuned included `n_estimators` that specifies the number of trees, `max_depth` that determines the depth of each tree, and `learning_rate` to prevent overfitting [12], as specified in the code implementation. The optimal hyperparameter configuration was then used in the final XGBoost model.

F. LightGBM Hyperparameter Tuning

To improve the LightGBM model, we selected and evaluated a set of key hyperparameters to balance performance and overfitting. This included `learning_rate = 0.05`, `num_leaves = 64`, and sampling parameters `feature_fraction = 0.8` and `bagging_fraction = 0.8`. We also applied regularization using `lambda_l1 = 0.1` and `lambda_l2 = 1.0`, and used early stopping with `stopping_rounds = 50` for up to `num_boost_round = 1000`. The best iteration on the validation set was then used to train the final model on the combined training and validation data.

G. Neural Network Hyperparameter Tuning

As for the Neural Network model, we manually tested different epochs values, such as 50, 100, and 120, as well as different batch size values, such as 16, 32, and 64. After experimenting with these configurations, the final epoch and batch size used that allowed us to obtain the best results after evaluating on the validation data were 120 and 32, respectively.

H. Convolutional Neural Network Hyperparameter Tuning

Furthermore, for the Convolutional Neural Network, we experimented with several values of L2 regularization. Initially, when no regularization was applied, the model achieved an accuracy of 64.16%. When we added L2 regularization, the accuracy of the model significantly improved. As shown in Table I, which summarizes the different values of L2 regularization experimented with and their corresponding results. The best performance was obtained using L2 regularization with a value of 0.01. Additionally, we manually tested with several epochs and batch sizes to obtain the best model configuration, where the final selected values for epochs and batch sizes are 50 and 32, respectively.

TABLE I
PERFORMANCE OF DIFFERENT L2 REGULARIZATION VALUES

L2 Value	Validation Accuracy (%)
No regularization	64.16
0.1	66.01
0.01	68.88
0.001	66.64

V. DISCUSSION

In this section, the models performances is discussed based on their results.

The Decision Tree model showed a low accuracy when evaluated on the validation set, causing it to have the poorest results compared to the other models we have experimented with. To analyze why we obtain bad results, we used our model to predict on the training data. The results show that the prediction on training data had 99.2% accuracy. The big gap between the prediction on the training data and the prediction on the validation data shows that the problem is likely due to overfitting.

Similarly, the Logistic Regression model initially achieved a low accuracy value, however, after adding polynomial features of degree 2, the model's accuracy improved significantly, achieving the best performance obtained with this approach. This suggests that the data is nonlinear, requiring additional polynomial features to capture the relationships [13].

The Random Forest model demonstrated an improvement compared to the earlier baseline models, achieving an accuracy of 58.9% on the validation set. This shows that using multiple decision trees helped reduce the overfitting problem seen in the single Decision Tree model.

Using the Gradient Boosting model, we achieved a good accuracy and a balanced trade-off between variance and bias. The model was not affected by feature engineering, therefore, it was able to learn effectively from the original features. However, the current features may still be insufficient for the model to fully capture the underlying patterns in the data [14].

To improve the results of the Gradient Boosting Model, we built an XGBoost model, as it is an optimized version of the classical gradient boost model [4]. After tuning the hyperparameters of our XGBoost model using randomized search to efficiently obtain the optimal hyperparameter values, the model's accuracy increased compared to the baseline version,

achieving the second highest accuracy we have obtained so far among the other models we have experimented with, such as the Neural Network and the Random Forest models. This is likely because it efficiently builds a tree boosting system that handles sophisticated patterns in data while using regularization to prevent overfitting [4]. However, its performance still does not surpass the performance of the CNN model.

Surprisingly, after evaluating the Neural Network model, it did not achieve the strong results we expected, especially since it is generally known to capture complex relationships and patterns in nonlinear data [5], such as the one used in this project. Although experiments were done on different epochs, batch sizes and dense layers, and feature engineering was applied, the model still achieved mediocre results. This is likely due to the highly imbalanced data, which favors column move 3.

The LightGBM model achieved an accuracy of 64.09%, performing better than the Random Forest model. This model was able to learn complex patterns more efficiently through its boosting approach, which builds trees step by step to improve prediction accuracy. It handled the multiclass classification task well and showed somewhat good generalization, making it the third best-performing model implemented in this project [7].

After evaluating the performance of the models we implemented, we decided to implement a Convolutional Neural Network (CNN), as we expected it to achieve a higher performance compared to the other models after converting the data into a 6×7 grid. During its development, we noticed that the CNN's performance significantly improved after applying L2 regularization. This indicates that there is a need for regularization to prevent overfitting, which is a problem we have commonly seen in the other models due to the characteristics of the given data. Moreover, the CNN achieved the best performance out of all the models we have implemented, which is a strong indication that CNNs are efficient in capturing features not only in images, but also when the data can be represented as a grid.

Overall, the dataset shows a clear imbalance in data, indicating a nonlinear complex pattern. After building several models and comparing their results on the validation set, CNN achieved the best performance, followed by the gradient boosting algorithms such as the Gradient Boosting model, XGBoost, and the LightGBM model. This indicates that both the gradient boosting algorithms and the CNN have a good ability of capturing complex, nonlinear relationships in data.

VI. Conclusion

In conclusion, this project explored different machine learning algorithms to predict the next move in the Connect Four game. The results showed that tree-based ensemble models such as Gradient Boosting, XGBoost, and LightGBM performed better than simpler models like Decision Tree and Logistic Regression. Among all models, the Convolutional Neural Network (CNN) achieved the highest validation accuracy of 68.88%, outperforming the tuned XGBoost and LightGBM models. Although the Neural Network model achieved moderate results, it was still affected by the data imbalance. Overall, the experiments showed strong predictive performance, and the results can be further improved by using more informative features and a larger dataset.

References

[1] M. Böck, "Strongly Solving 7×6 Connect-Four on Consumer Grade Hardware," *arXiv preprint* arXiv:2507.05267, 2025. [Online]. Available: https://arxiv.org/abs/2507.05267.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[3] J. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics*, vol. 29, Nov. 2000.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2016, pp. 785–794.

[5] R. Qamar and B. Zardari, "Artificial Neural Networks: An Overview," *Mesopotamian Journal of Computer Science*, vol. 2023, pp. 130–139, Aug. 2023, doi: 10.58496/MJCSC/2023/015.

[6] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv:1412.6980, 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Proc. 31st Conf. Neural Inf. Process. Syst. (NeurIPS), 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf.

[8] L. Alzubaidi, J. Zhang, A. J. Humaidi, et al., "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions," *Journal of Big Data*, vol. 8, no. 53, 2021. doi: 10.1186/s40537-021-00444-8.

[9] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, "How Does Batch Normalization Help Optimization?," arXiv:1805.11604, 2019. [Online]. Available: https://arxiv.org/abs/1805.11604

[10] scikit-learn Developers, "*DecisionTreeClassifier — scikit-learn 1.7.2 Documentation*," Available: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html. Accessed: Oct. 31, 2025.

[11] J. Bergstra, Y. Bengio, and Y. Rachmad, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, Mar. 2012.

[12] XGBoosting, "Most Important XGBoost Hyperparameters to Tune." [Online]. Available: https://xgboosting.com/most-important-xgboost-hyperparameters-to-tune/.

[13] X. Wan, "The Influence of Polynomial Order in Logistic Regression on Decision Boundary," *IOP Conference Series: Earth and Environmental Science*, vol. 267, p. 042077, May 2019, doi: 10.1088/1755-1315/267/4/042077.

[14] J. Heaton, "An empirical analysis of feature engineering for predictive modeling," in *Proceedings of SoutheastCon 2016*, pp. 1–6, Mar. 2016, doi: 10.1109/SECON.2016.7506650.

APPENDIX

This section provides the code implementation of our project, including the feature engineering techniques and models implementation.

Importing the needed libraries:

```
#import the needed libraries:
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    roc_curve
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import classification_report
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import plot_tree
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import accuracy_score, confusion_matrix
from xgboost import XGBClassifier
from sklearn.ensemble import RandomForestClassifier
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout
from sklearn.pipeline import Pipeline
from tensorflow.keras.optimizers import Adam
from scikeras.wrappers import KerasClassifier as SKKerasClassifier
from sklearn.ensemble import GradientBoostingClassifier
```

Loading the dataset:

```
#Loading the train,test, and validation datasets:
train_data=pd.read_csv('/content/drive/MyDrive/ML project/csc462-connect-4/train.csv')
test_data=pd.read_csv('/content/drive/MyDrive/ML project/csc462-connect-4/test.csv')
validation_data=pd.read_csv('/content/drive/MyDrive/ML project/csc462-connect-4/val.csv')
#print to check:
train_data.head()

#convert the turn column of -1 and 1 into 0 and 1 instead, (numerical to binary feature)( commented out as
    it had no use)
#train_data['turn']=train_data['turn'].map({-1:0,1:1})
#validation_data['turn']=validation_data['turn'].map({-1:0,1:1})
#test_data['turn']=test_data['turn'].map({-1:0,1:1})

#divide train data into input and label:
X_train=train_data.drop('label_move_col',axis=1)
y_train=train_data['label_move_col']
X_val=validation_data.drop('label_move_col',axis=1)
y_val=validation_data['label_move_col']
X_test = test_data.drop('id', axis=1)
```

Decision Tree Model Implementation:

```
#Decision Tree Model
#Step 1: creating the model:
DecisionTree_model = DecisionTreeClassifier(random_state=42, class_weight="balanced")


#Step 2: model training
DecisionTree_model.fit(X_train,y_train)

#Step 3: prediction using validation data:
y_pred_val=DecisionTree_model.predict(X_val)

#Step 4: Evaluate the model
print(classification_report(y_val,y_pred_val))
print("Baseline Accuracy: ", accuracy_score(y_val,y_pred_val))

#Step 5: print the confusion matrix
Confusion_Matrix=confusion_matrix(y_val,y_pred_val)
```

```
display=ConfusionMatrixDisplay(confusion_matrix=Confusion_Matrix,display_labels=DecisionTree_model.classes_
    )
display.plot(cmap='Blues')
plt.title('Decision Tree Confusion Matrix')
plt.show()
```

Logistic Regression Model Implementation:

```
#Logistic Regression Model
feature_list = [f"p{i}" for i in range(1, 43)] + ['turn']
train_x = train_data[feature_list]
train_y = train_data['label_move_col']
validation_x = validation_data[feature_list]
validation_y = validation_data['label_move_col']

# Polynomial Features degree 2
poly = PolynomialFeatures(degree=2, include_bias=False)
train_x_poly = poly.fit_transform(train_x)
validation_x_poly = poly.transform(validation_x)

# Logistic Regression with regularization
# C_values = [0.01, 0.1, 1, 10, 100], were tested, with C=100 giving the best accuracy
#for c in C_values:
log_reg = LogisticRegression(
    C=100,                # regularization strength
    solver='lbfgs',
    max_iter=1000,
    class_weight='balanced',
    random_state=42
)
log_reg.fit(train_x_poly, train_y)
y_pred = log_reg.predict(validation_x_poly)
acc = accuracy_score(validation_y, y_pred)
print(f"C=100 --> Accuracy={acc}")

# Confusion Matrix
con_mat = confusion_matrix(validation_y, y_pred)
display=ConfusionMatrixDisplay(confusion_matrix=con_mat,display_labels=log_reg.classes_)
display.plot(cmap='Blues')
plt.title('Logistic Regression Confusion Matrix')
plt.show()
```

Random Forest Model Implementation:

```
#Random Forest Model
from sklearn.ensemble import RandomForestClassifier
# try a few manual settings (for faster runtime we kept the best result and turned the others into comments
    )
rf_settings = [
    {"n_estimators": 300, "max_depth": None},  # best so far with Accuracy: 0.58947
    # {"n_estimators": 350, "max_depth": None},
    # {"n_estimators": 100, "max_depth": None},
    # {"n_estimators": 300, "max_depth": 20},
]

# testing loop to get the best accuracy
best_acc = -1.0
best_params = None

for i, params in enumerate(rf_settings, 1):
    print(f"\n Random Forest Run {i}: {params} ")
    rf = RandomForestClassifier(
        n_estimators=params["n_estimators"],
        max_depth=params["max_depth"],
        max_features="sqrt",
        min_samples_split=2,
        min_samples_leaf=1,
        class_weight=None,
        n_jobs=-1,
        random_state=42,
        bootstrap=True,
    )
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_val)

    acc = accuracy_score(y_val, y_pred)
    print("Validation Accuracy:", acc)
```

```
    print(classification_report(y_val, y_pred, digits=3))

    cm = confusion_matrix(y_val, y_pred)
    ConfusionMatrixDisplay(confusion_matrix=cm).plot(cmap="Blues")
    plt.title("Random Forest Confusion Matrix")
    plt.show()

    if acc > best_acc:
        best_acc = acc
        best_params = params

print("\nBest of the settings:", best_params, "with acc =", best_acc)

# train final model with the best setting from the loop on all labeled data
final_rf = RandomForestClassifier(
    n_estimators=best_params["n_estimators"],
    max_depth=best_params["max_depth"],
    max_features='sqrt',
    min_samples_leaf=1,
    bootstrap=True,
    class_weight=None,
    random_state=42,
    n_jobs=-1
)

# we combined training set with validation for the final model to train on
full_X =pd.concat([X_train, X_val], axis=0)
full_y = pd.concat([y_train, y_val], axis=0)
final_rf.fit(full_X, full_y)

# predict and save as required: id + label_move_col
test_pred = final_rf.predict(X_test)
submission = pd.DataFrame({
    "id": test_data['id'] if 'id' in test_data.columns else np.arange(1, len(test_pred) + 1, dtype=int),
    "label_move_col": test_pred
})
submission.to_csv("submission.csv", index=False)
print("Done: submission.csv")
```

Enhancing the dataset with Feature Engineering for advanced models:

```
#Enhancing the dataset with feature engineering for advanced models
# Get the board columns  p 1 p42
board_cols = [col for col in X_train.columns if col.startswith('p')]

# Multiply each board cell by its turn value
X_train[board_cols] = X_train[board_cols].multiply(X_train['turn'], axis=0)
X_val[board_cols]   = X_val[board_cols].multiply(X_val['turn'], axis=0)
X_test[board_cols]  = X_test[board_cols].multiply(X_test['turn'], axis=0)

# Drop turn column
X_train = X_train.drop(columns=['turn'])
X_val   = X_val.drop(columns=['turn'])
X_test  = X_test.drop(columns=['turn'])

# Display a few transformed samples
print("\nSample transformed training rows:")
print(X_train.head())
```

XGBoost Model Implementation:

```
#Modified XGBoost model
#hyperparameter tuning to obtain better results than the baseline

param_distribution = {
    'n_estimators': [200,250,300,350,400],
    'max_depth': [3,7,10,15,20],
    'learning_rate': [0.01,0.1,0.2],
    'subsample': [0.5, 0.7,1],
    'colsample_bytree': [0.5, 0.7],
    'lambda': [0.01,0.1, 1.0, 10.0]
    }
XGBoostModified= XGBClassifier()
Rsearch = RandomizedSearchCV(estimator=XGBoostModified, param_distributions=param_distribution, scoring='
    accuracy',cv=3, random_state=42)
Rsearch.fit(X_train, y_train)
print("Best parameters:", Rsearch.best_params_)
```

```

XGBoostModified=XGBClassifier(**Rsearch.best_params_,random_state=42,objective="multi:softprob")
XGBoostModified.fit(X_train,y_train)
y_pred_val=XGBoostModified.predict(X_val)
#results:
print(classification_report(y_val,y_pred_val))
#confusion matrix:
cm = confusion_matrix(y_val, y_pred_val)
ConfusionMatrixDisplay(confusion_matrix=cm).plot(cmap="Blues")
plt.title(f"Confusion Matrix of XGBoost")
plt.show()
#accuracy
XGMaccuracy=accuracy_score(y_val,y_pred_val)
print("Modified XGBoost accuracy: % ",XGMaccuracy*100)
```

Neural Network Model Implementation:

```
#Neural Network Model
tf.keras.utils.set_random_seed(42)

#Step 1: creating the neural network model archeticture:
NNmodel = keras.Sequential([
    Dense(128, input_shape=(X_train.shape[1],), activation='relu'),    #activation function is ReLU or
    Dropout(0.2),                    #dropout layer to prevent overfitting
    Dense(64, activation='relu'),
    Dropout(0.2),  #dropout layer to prevent overfitting
    Dense(32, activation='relu'),
    Dropout(0.2),  #dropout layer to prevent overfitting
    Dense(7, activation='softmax') #output layer
])


#Step 2: compile the model
NNmodel.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

#Step 3: Train the model
history=NNmodel.fit(X_train, y_train, epochs=120, batch_size=32, validation_data=(X_val, y_val))

#Step 4: evaluate the model
val_loss, val_acc = NNmodel.evaluate(X_val, y_val)
print('Neural Network accuracy: %', val_acc*100)
#predict and create the confusion matrix
y_pred=NNmodel.predict(X_val)
y_pred=np.argmax(y_pred,axis=1)
ConfusionMatrix=confusion_matrix(y_val,y_pred)
ConfusionMatrixDisplay(confusion_matrix=ConfusionMatrix).plot(cmap="Blues")
plt.title('Neural Network Confusion Matrix')
plt.show()
```

Gradient Boosting Model Implementation:

```

#Gradient Boosting Model
best_gb = GradientBoostingClassifier(
    n_estimators=400,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,            # reduces variance
    min_samples_split=5,      # prevents overfitting
    min_samples_leaf=2,
    random_state=42
)

# Train the model on the training data
best_gb.fit(X_train, y_train)

# Predict on validation data
y_pred_val = best_gb.predict(X_val)
acc = accuracy_score(y_val, y_pred_val)
print("Gradient Boosting accuracy: %'", acc)
cm = confusion_matrix(y_val, y_pred_val)
ConfusionMatrixDisplay(confusion_matrix=cm).plot(cmap="Blues")
plt.title('Gradient Boosting Confusion Matrix')
plt.show()
```

LightGBM Model Implementation:

```
#LightGBM Model
import lightgbm as lgb

# Prepare datasets
train_data = lgb.Dataset(X_train, label=y_train)
val_data   = lgb.Dataset(X_val, label=y_val, reference=train_data)

# Parameters
params = {
    'objective': 'multiclass',
    'num_class': 7,
    'metric': 'multi_logloss',
    'learning_rate': 0.05,
    'num_leaves': 64,
    'feature_fraction': 0.8,
    'bagging_fraction': 0.8,
    'bagging_freq': 4,
    'min_data_in_leaf': 50,
    'max_depth': -1,
    'lambda_l1': 0.1,
    'lambda_l2': 1.0,
    'random_state': 42
}

# Correct way to handle early stopping (via callbacks)
callbacks = [
    lgb.early_stopping(stopping_rounds=50),
    lgb.log_evaluation(period=50)
]

# Train model
model_lgb = lgb.train(
    params=params,
    train_set=train_data,
    valid_sets=[val_data],
    num_boost_round=1000,
    callbacks=callbacks
)

# Predictions
y_pred = np.argmax(model_lgb.predict(X_val, num_iteration=model_lgb.best_iteration), axis=1)
val_acc = accuracy_score(y_val, y_pred)
print(f"LightGBM accuracy:%  {val_acc:.4f}")

# Retrain on Full Data (train + val)
X_full = np.concatenate([X_train, X_val])
y_full = np.concatenate([y_train, y_val])

full_data = lgb.Dataset(X_full, label=y_full)

final_model = lgb.train(
    params=params,
    train_set=full_data,
    num_boost_round=model_lgb.best_iteration  # use best iteration from validation
)

test_pred = np.argmax(final_model.predict(X_test), axis=1)
# Plot confusion matrix
cm = confusion_matrix(y_val, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix - LightGBM Validation Set")
plt.show()
```

CNN data feature engineering:

```
 #Loading the train,test, and validation datasets:
train_data=pd.read_csv('/content/drive/MyDrive/ML project/csc462-connect-4/train.csv')
test_data=pd.read_csv('/content/drive/MyDrive/ML project/csc462-connect-4/test.csv')
validation_data=pd.read_csv('/content/drive/MyDrive/ML project/csc462-connect-4/val.csv')

#divide train data into input and label:
X_train=train_data.drop('label_move_col',axis=1)
y_train=train_data['label_move_col']
X_val=validation_data.drop('label_move_col',axis=1)
```

```
y_val=validation_data['label_move_col']
X_test = test_data.drop('id', axis=1)

def cnn_feature_engineering(data):
  data=data.copy() #copy the data
  # Get the board columns  p 1 p42
  grid_cols = [col for col in data.columns if col.startswith('p')]
  # Multiply board cell by its turn value
  data[grid_cols] = data[grid_cols].multiply(data['turn'], axis=0)
  # Drop turn column
  data = data.drop(columns=['turn'])

  grid=data[grid_cols].values.reshape(-1,6,7) #change the shape into a grid, 6 rows and 7 columns

#I converted the board into 3 different channels, channel 1 for the current player, channel 2 for the oponnent, and channel 3 for the empyt cell
#So the CNN can understand the shape of the grid better
  current_player= (grid==1).astype("float32")
  opponent_player=(grid==-1).astype("float32")
  empty_cell=(grid==0).astype("float32")

  x_gird= np.stack([current_player,opponent_player,empty_cell],axis=-1) # stack the channels into a tensor
  return x_gird

#reshape for CNN
X_train_CNN=cnn_feature_engineering(X_train)
X_val_CNN=cnn_feature_engineering(X_val)
X_test_CNN=cnn_feature_engineering(X_test)
```

CNN Implementation:

```
def createCNN():
  #regularization
 l2_reg=keras.regularizers.l2(0.01)
 model=Sequential([
      Conv2D(32,kernel_size=(3,3),activation='relu', padding='same', input_shape=(6,7,3),kernel_regularizer=l2_reg), #input shape is (6,7,3) because we have 6 rows, 7 columns, and 3 channels (from FE)
      BatchNormalization(),

      Conv2D(32,kernel_size=(3,3),activation='relu',padding='same',kernel_regularizer=l2_reg),
      BatchNormalization(),

      MaxPooling2D(pool_size=(2,2)),


      Conv2D(64,kernel_size=(3,3),activation='relu',padding='same',kernel_regularizer=l2_reg),
      BatchNormalization(),

      Conv2D(64,kernel_size=(3,3),activation='relu',padding='same',kernel_regularizer=l2_reg),
      BatchNormalization(),


      Flatten(),
      Dense(128,activation='relu'),
      Dropout(0.4),
      Dense(64,activation='relu'),
      Dropout(0.3),
      Dense(7,activation='softmax'),

  ])
 model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
 model.summary()
 return model
```

CNN Training:

```


#fixed seed to reproduce
tf.random.set_seed(42)
np.random.seed(42)
CNN_model= createCNN()
lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3) #learning rate will be reduced by half if no improvement seen after 3 epochs
early_stopping=EarlyStopping(monitor='val_loss',patience=8, restore_best_weights=True)
```

```
history=CNN_model.fit(X_train_CNN, y_train, validation_data=(X_val_CNN, y_val), batch_size=32, epochs=50,
    callbacks=[lr_scheduler,early_stopping],verbose=1)

val_loss, val_acc = CNN_model.evaluate(X_val_CNN, y_val)
print('CNN accuracy: %', val_acc*100)
```

CNN Results:

```
y_pred=CNN_model.predict(X_val_CNN)
y_pred=np.argmax(y_pred,axis=1)
ConfusionMatrix=confusion_matrix(y_val,y_pred)
ConfusionMatrixDisplay(confusion_matrix=ConfusionMatrix).plot(cmap="Blues")
plt.title(' CNN Confusion Matrix')
plt.show()

# predict and save
test_pred_probs = CNN_model.predict(X_test_CNN)
test_pred= np.argmax(test_pred_probs, axis=1)
submission = pd.DataFrame({
    "id":test_data["id"] if "id" in test_data.columns else np.arange(1, len(test_pred) + 1, dtype=int),
    "label_move_col": test_pred
})
submission.to_csv("submission.csv", index=False)
print("Done: submission.csv")
```